

GraphOne Data Intelligence Architecture

1. Scale Strategy (500k+ Records)

To process 500,000+ records without manual intervention:

- **Distributed Crawler Nodes:** We will package the Playwright/Node.js scrapers into Docker containers and orchestrate them via Kubernetes (or AWS ECS).
- **Message Queues:** A central message broker (e.g., Kafka or RabbitMQ) will manage URLs to be scraped. Workers will pull URLs, scrape them, and publish the raw HTML back to the queue.
- **Asynchronous Processing:** The crawler leverages Playwright Async to handle multiple browser contexts concurrently.

2. Handling Context Window Overflows (413s) & Rate Limits (429s)

- **429 Handling (Rate Limits):** The `LLMOrchestrator` implements an exponential backoff with jitter strategy. If Gemini Flash returns a 429, we wait `delayMs + jitter` before retrying. If retries exhaust, we fall back to Groq Llama 3, then DeepSeek.
- **413 Handling (Payload Too Large):** Raw HTML from pages is often bloated. Before sending to the LLM:
 1. We strip `<script>`, `<style>`, `<nav>`, and `<footer>` tags using Cheerio/BeautifulSoup equivalents.
 2. If the text exceeds the token limit (e.g., >8000 tokens for Groq), we implement an **Intelligent Chunking Strategy**, splitting text into overlapping chunks and performing Map-Reduce LLM extraction.

3. Freshness Tracking (Never process twice)

To ensure we only process jobs and news from the last 24 hours exactly once:

- **URL & Hash Cache:** We maintain an in-memory Redis cache (or Memcached) storing the SHA-256 hash of scraped URLs and content snippets.
- **Time-to-Live (TTL):** The Redis keys have a 24-hour TTL. If a URL/Hash exists in Redis, the crawler skips it immediately, preventing redundant LLM calls.
- **Date Normalization heuristics:** We use regex and date-fns to parse relative dates ("2 hours ago"). If no date is found, we fall back to checking if the entity hash exists in our DB.

4. Storage Strategy

- **Primary Database (PostgreSQL):** PostgreSQL is chosen for robust relational integrity. Startups, Products, and Jobs have strict JSON schemas and relationships (e.g., a Product belongs to a Startup).
- **Vector Storage (Pinecone):** For mapping complex relationships and enabling semantic search (e.g., finding "Similar Startups"), we will embed the startup descriptions and store them in Pinecone.
- **Entity Resolution Storage:** The canonical seed list acts as a source of truth table in Postgres. Extracted entities are fuzzy-matched against this table.